

STAT5243-Project-1

This is repository for Columbia University STAT GR2543 Project 1 # Project 1 Report — IBM Job Postings Data Pipeline (Scraping → Cleaning → EDA → Preprocessing → Feature Engineering)

Course: STAT5243 — Project 1 **Dataset:** IBM job postings (web-scraped) **Team Members:** Kevin Ma, Shuzhi Yang, Baixuan Chen, Carrie Feng **Date:** Feb 2026

1. Introduction and Dataset Description

This project builds an end-to-end data pipeline on a real-world dataset: acquiring data from an open source via web scraping, then cleaning, exploring, preprocessing, and engineering features to make the dataset suitable for exploratory analysis and future predictive modeling. The goal is to demonstrate practical skills in handling messy data, documenting methodology clearly, and producing actionable insights from raw information.

1.1 Dataset Overview

Our dataset consists of **478 IBM job postings** collected from IBM's job listings during the project period. Each row represents a job posting and includes structured fields describing the role, location, posting date, and salary range.

Shape: 478 rows × 11 columns

1.2 Data Dictionary (Columns)

Column	Type (raw)	Meaning
job_title	text	Job title string shown in the posting
job_id	text/id	Unique identifier for the posting
date_posted	text	Posting date (e.g., 30-Jan-2026)
state_province	text	Location field (often one or multiple states)
area_of_work	text	Job function category (e.g., Consulting, Software Engineering)
min_salary	text/number-like	Minimum annual salary (string formatted)
max_salary	text/number-like	Maximum annual salary (string formatted)

Column	Type (raw)	Meaning
<code>position_type</code>	text	Professional / Internship / Entry Level / etc.
<code>required_education</code>	text	Required education level
<code>preferred_education</code>	text	Preferred education level (often missing)
<code>preferred_technical_experience</code>	text	Unstructured text describing desired experience/skills

1.3 Why This Dataset?

Job postings are a strong real-world example of semi-structured data: salary fields are formatted inconsistently, location fields may contain multiple values, and the “preferred technical experience” field is unstructured text. This creates meaningful opportunities for advanced cleaning and feature engineering, aligning with the project’s emphasis on real-world data challenges.

2. Data Acquisition Methodology

2.1 Source and Collection Strategy

Kevin Ma acquired the dataset through **web scraping** IBM’s public job listings. The scraping workflow followed a structured approach:

1. Identify relevant IBM job listing pages (search results + individual job pages).
2. Iterate through listings (pagination / repeated page loads).
3. Extract structured fields for each posting (title, job id, date, location, salary range, area of work, education, and technical experience).
4. Store results in a CSV file (`ibm_jobs.csv`) for downstream processing.

This acquisition approach satisfies the project requirement to obtain data from sources such as web scraping.

2.2 Acquisition Challenges (and how we handled them)

- **Pagination / repeated listings:** Web job portals often show repeating listings across pages. We relied on `job_id` as a stable key for deduplication downstream.
- **Inconsistent formatting:** Salary appeared as formatted text (commas/decimals), and locations sometimes contained multiple states. These were handled during cleaning and feature engineering.

- **Missing fields:** Not all postings include preferred education or technical experience. Missingness was tracked and handled in later steps.
-

3. Cleaning and Preprocessing Steps

This section documents the work required to turn raw scraped output into consistent, analysis-ready tables. The assignment explicitly emphasizes detecting inconsistencies, enforcing uniform formatting, handling duplicates, outliers, and missingness.

3.1 Cleaning (Kevin Ma)

Key cleaning actions:

- **Deduplication:** Removed duplicate job postings using `job_id` as a unique key (**7 duplicated `job_ids`** observed in the raw file).
- **Uniform formatting:** Trimmed whitespace in string fields to avoid category fragmentation (e.g., "Texas" vs " Texas").
- **Type correction:** Converted salary fields stored as strings into numeric values; parsed `date_posted` into a datetime.
- **Missing value handling (baseline):**
 - Preserved missingness in optional fields so later preprocessing and modeling can treat “missing” meaningfully rather than overwriting it prematurely.

3.2 Preprocessing (Baixuan Chen)

Preprocessing focuses on preparing features for analysis/modeling (without necessarily creating new meaning). Consistent with the project requirements, preprocessing includes steps such as normalization/standardization and encoding categorical variables.

Preprocessing approach:

- **Missing values**
 - For structured categorical fields (e.g., education), missing values were assigned a consistent placeholder category (e.g., "**Unknown**"), so they remain usable in grouped analysis.

- For unstructured text (technical experience), missing values were converted to empty strings to avoid errors in text-based processing.
- **Numeric conversions**
 - Salary strings were cleaned (removing commas and symbols) and converted to numeric.
- **Encoding strategy (model-ready)**
 - Categorical columns are suitable for one-hot encoding when modeling.
 - Numeric columns are suitable for scaling if used in regression models.

3.3 Missingness Summary (Raw Dataset)

To document data quality issues clearly, we measured missingness rates:

- **preferred_education**: ~25.7% missing
- **preferred_technical_experience**: ~19.2% missing
- **required_education**: ~2.5% missing
- **area_of_work**: ~1.0% missing

This confirms the dataset contains meaningful real-world incompleteness—especially in “preferred” fields that companies often omit.

4. Exploratory Data Analysis (EDA)

Shuzhi Yang performed the EDA to understand the distribution of job types, salary ranges, and relationships between role characteristics and compensation. The EDA aligns with project expectations: summary statistics, distribution plots, relationship exploration, and initial interpretations.

4.1 Salary Distribution Overview

After converting salaries to numeric:

- **Median midpoint salary (mid_salary): \$134,000**
- **Mean midpoint salary: ~\$141,138**
- **Minimum midpoint salary: \$35,500**
- **Maximum midpoint salary: \$342,500**

Salary ranges were often wide:

- **Median salary range (max - min): \$69,900**
- **Max salary range: \$216,000**
- Some postings had a **0 range** (min = max), suggesting a fixed-band listing.

Interpretation: compensation is right-skewed and varies strongly by role class and seniority.

4.2 Job Type Composition

Posting counts by `position_type`:

- **Professional: 273**
- **Internship: 103**
- **Entry Level: 92**
- **Administration & Technician: 10**

Interpretation: IBM postings in this scrape are dominated by professional roles, but internships and entry-level roles are also well represented, which creates a broad salary spread.

4.3 Salary by Position Type

Median midpoint salary by type:

- **Professional: \$174,000**
- **Entry Level: \$113,500**
- **Internship: \$103,650**
- **Administration & Technician: \$56,000**

Interpretation: compensation differences across posting type are large and consistent with expected labor market structure.

4.4 Area of Work Insights

Most common `area_of_work` categories included Consulting, Software Engineering, and Infrastructure & Technology. Salary medians varied substantially across these categories, suggesting job function is an important explanatory variable.

4.5 Location Field Complexity

The `state_province` column often included **multiple states** in one row (e.g., “Texas, Massachusetts, California”), which indicates flexible or multi-location postings. This motivated our later feature engineering on location structure.

5. Feature Engineering Process and Justification (Carrie Feng)

Feature engineering focuses on **creating meaningful new variables** that capture signal not directly represented in the raw columns. The project rubric specifically rewards advanced feature engineering that improves predictive potential through creativity and a deep understanding of feature principles.

5.1 Salary Structure Features (Turning “range” into signal)

Problem: Raw `min_salary` and `max_salary` are informative, but they do not explicitly represent compensation structure.

Engineered features:

- `mid_salary` = $(\text{min} + \text{max}) / 2$
- `salary_range` = $\text{max} - \text{min}$
- `salary_range_pct` = $(\text{max} - \text{min}) / \text{mid_salary}$
- `min_to_max_ratio` = min / max
- `log_mid_salary` = $\log(\text{mid_salary})$

Why this helps: Range width can proxy how flexible the role level is, and log transforms reduce skew and improve robustness for future modeling.

5.2 Time-Derived Features (Recency + seasonality)

Problem: `date_posted` is text; time patterns are not directly usable.

Engineered features:

- `days_since_posted` (relative to the latest posting in the dataset)
- `posted_month`, `posted_day_of_week`, `posted_is_weekend`

Why this helps: Hiring cycles and urgency may influence salary bands and role availability over time.

5.3 Location Structure Features (Multi-state postings + region)

Problem: `state_province` sometimes contains multiple states, which can't be modeled as a single clean category.

Engineered features:

- `states_list` (parsed list)
- `n_states_listed`
- `is_multi_state_posting`
- `primary_state` (first listed state)
- `primary_region` (coarse mapping: Northeast/Midwest/South/West)

Why this helps: It captures remote/flexible hiring structure and reduces high-cardinality geographic noise while preserving meaningful geographic patterns.

5.4 Education as Ordinal Signal (and “education gap”)

Problem: Education is not merely categorical; it has a natural order, and preferred vs required difference is meaningful.

Engineered features:

- `required_edu_level`, `preferred_edu_level` (ordinal mapping)
- `edu_gap_preferred_minus_required`
- `has_preferred_edu_specified`

Why this helps: More selective roles may correlate with higher compensation and different job families.

5.5 Job Title Parsing (Seniority + role family extraction)

Problem: Job title includes strong but hidden salary signals (e.g., Senior, Lead, Manager).

Engineered features (binary flags):

- Seniority: `is_intern`, `is_entry`, `is_senior`, `is_lead`, `is_manager`, `is_architect`
- Role family: `role_engineering`, `role_data`, `role_security`, `role_product`, etc.

Why this helps: These features are interpretable, strong predictors of pay, and reduce the need for manual labeling.

5.6 Skills + Experience from Text (Advanced unstructured feature extraction)

Problem: `preferred_technical_experience` is unstructured text.

Engineered features:

- Experience years parsed from text: `exp_years_min`, `exp_years_max`, `exp_years_any`
- Skill flags (examples): `skill_python`, `skill_sql`, `skill_aws`, `skill_azure`, `skill_kubernetes`, etc.
- `n_skills_mentioned` = total count of detected skill keywords

Why this helps: It converts qualitative requirements into quantitative features, enabling analysis like “how do cloud/ML skill mentions relate to salary?”

6. Summary of Key Findings

1. **Salary levels differ strongly by posting type.** Professional roles have much higher median pay than internships/entry level (professional median midpoint ~\$174k vs internships ~\$104k).
 2. **Salary ranges are wide and meaningful.** Many postings include broad compensation bands; range-based features capture this structure directly.
 3. **Job title seniority correlates with pay.** Titles containing “Senior” tend to have higher median midpoint salary than entry/intern indicators, supporting the value of title-parsed features.
 4. **Location is complex, not single-valued.** Many postings list multiple states, so “multi-location flexibility” is a real structural feature, not noise.
 5. **Preferred fields are frequently missing.** Preferred education and technical experience are often omitted, so missingness handling and missing-indicator logic are important for reliability.
-

7. Challenges Faced and Future Recommendations

7.1 Challenges Faced

- **Inconsistent formatting (salary + location):** Salary appeared as formatted strings, and locations sometimes contained multiple states in one field.
- **Missingness in key descriptive fields:** Preferred education and technical experience were missing for a substantial fraction of postings.

- **High-cardinality categoricals:** Locations and titles can explode the number of categories if used naively.
- **Unstructured text complexity:** Skill and experience extraction can be noisy because different postings describe the same skills in different ways.

7.2 Recommendations for Future Work

1. **Improve location modeling:** Add external cost-of-living indices or metro-area mapping to better interpret geographic salary differences.
 2. **More robust NLP features:** Use TF-IDF, embeddings, or topic modeling to capture skill patterns beyond keyword lists.
 3. **Model validation:** Train a baseline regression model (e.g., Ridge/Lasso) to quantify how engineered features improve predictive performance.
 4. **Scraping expansion:** Collect additional fields (remote indicator, job level, department, benefits) and increase time coverage to reduce sampling bias.
 5. **Better experience parsing:** Enhance regex rules to detect ranges like “3–5 years” and normalize alternative phrases (“years of experience,” “yrs exp”).
-

8. Each Member’s Contribution

- **Kevin Ma — Data Acquisition & Initial Cleaning**
 - Web scraping IBM job postings
 - Compiled raw dataset into CSV
 - Performed initial cleaning and formatting corrections
- **Shuzhi Yang — Exploratory Data Analysis (EDA)**
 - Conducted summary statistics and visual exploration
 - Identified key distributions and relationships (salary vs job type, job function, location complexity)
- **Baixuan Chen — Preprocessing**
 - Implemented preprocessing workflow: handling missingness, type conversions, and preparing variables for downstream modeling (encoding/scaling readiness)
- **Carrie Yan Yin Feng — Feature Engineering & Written Report**
 - Implemented engineered features (salary structure, time, location structure, education ordinal encoding, title parsing, and text-derived skill/experience features)

- Completed written report
-

9. Conclusion

This project demonstrates a complete data pipeline from acquisition to engineered, analysis-ready features. The dataset’s real-world messiness—missing preferred fields, multi-location strings, and unstructured technical experience—made it a strong candidate for advanced feature engineering. The final engineered dataset is more interpretable, more structured for analysis, and better positioned for future predictive modeling tasks, meeting the project’s objectives and evaluation criteria.